

Probabilistic Recognition of Almost Primes by Machine Learning

Luis Gomez¹ and Yangbo Ye ^{*1}

¹Department of Mathematics, University of Iowa, Iowa City, IA 52242-1419, United States

August 14, 2026

Abstract

The probabilistic recognition of almost primes without explicit factorization is a natural problem in computational number theory. We investigate whether machine learning can distinguish r -almost primes, $\Omega(n) = r$, for $r = 2, \dots, 8$, under their natural distribution. Using the integer-vector encoding framework of Wang and Ye (2025), we train random forest (RF) and logistic regression (LR) classifiers on integers near 2^{44} and evaluate them, without retraining, on substantially larger integers near 2^{88} , 2^{132} , and 2^{176} .

Three principal findings emerge. First, RF classifiers are successfully trained for all seven r -almost prime classification problems, consistently outperforming both LR and the natural random-guessing baseline at the training scale. Second, genuine cross-scale generalization is observed only for semiprimes and 3-almost primes. Third, comparisons with LR and random guessing indicate that semiprimes and 3-almost primes possess statistical regularities that are considerably easier to recognize than those associated with higher-order almost primes. These results provide computational evidence that lower-order almost primes exhibit distinctive arithmetic structure worthy of further theoretical investigation.

Keywords: almost primes, semiprimes, computational number theory, probabilistic recognition, machine learning.

1 Introduction

Prime numbers and almost primes are among the most fundamental objects in number theory. An integer n is called an r -almost prime if it has exactly r prime factors counted with multiplicity, that is, $\Omega(n) = r$, where $\Omega(n)$ denotes the total number of prime divisors of n . In particular, primes are precisely the 1-almost primes, while semiprimes are the 2-almost primes. The distribution of almost primes has been studied extensively since the pioneering work of Landau [10, 11] and remains central to analytic number theory, sieve methods (cf. Halberstam and Richert [6]), additive number theory, and the study of multiplicative functions [7, 16]. Results of Hardy and Ramanujan [8] and the Erdős–Kac theorem [4] further reveal the remarkable statistical behavior of $\Omega(n)$, making almost primes a natural testing ground for probabilistic and statistical methods.

Although primality can now be determined deterministically in polynomial time by the celebrated AKS algorithm [1], no comparable deterministic method is known for recognizing r -almost primes without explicitly factoring the integer. Integer factorization remains computationally difficult for large inputs, and semiprimes occupy a particularly important position because of their

*Corresponding author: yangbo-ye@uiowa.edu

role in the RSA cryptosystem [13]. More generally, the problem of identifying almost primes from arithmetic information without complete factorization is of independent interest in computational number theory. Rather than replacing exact algorithms, one may ask whether machine learning can exploit statistical regularities in the arithmetic structure of integers to distinguish r -almost primes from other integers with nontrivial accuracy.

Closely related classification problems have also been studied from an algorithmic perspective. For example, Booker, Hiary, and Keating [2] developed efficient algorithms for detecting squarefree numbers. These results further illustrate that recognizing arithmetic properties of integers remains an active area of computational number theory.

Recent work has demonstrated that machine learning can successfully recognize several arithmetic properties of integers. Qin and Ye [12] applied random forests and neural networks to compute values of the Möbius function, while Wang and Ye [19, 20] introduced integer-vector encodings that allow machine learning algorithms to operate on integers far beyond the exact range of floating-point arithmetic. Using these encodings, they trained random forest classifiers to distinguish semiprimes from 3-almost primes on balanced datasets. Their work suggests that machine learning is capable of capturing arithmetic patterns that are difficult to describe explicitly.

The purpose of the present paper is to investigate a much broader question. Instead of focusing only on semiprimes, we consider the probabilistic recognition of r -almost primes for every $r = 2, \dots, 8$. For each fixed r , we study the binary classification problem of distinguishing integers satisfying $\Omega(n) = r$ from integers with $\Omega(n) \in \{2, \dots, 8\} \setminus \{r\}$, using naturally occurring datasets rather than artificially balanced ones. We train random forest (RF) and logistic regression (LR) classifiers on integers near 2^{44} using the integer-vector encodings of Wang and Ye, and then evaluate their performance both at the training scale and, more importantly, on much larger integers near 2^{88} , 2^{132} , and 2^{176} without retraining.

Our experiments reveal three principal findings. First, random forest classifiers can be successfully trained for all seven r -almost prime classification problems, $r = 2, \dots, 8$, consistently outperforming random guessing on the training data. Second, only the classifiers for semiprimes and 3-almost primes retain their predictive power when applied to substantially larger integers, indicating genuine cross-scale generalization. Finally, comparisons of the harmonic performance metric P_4 with both random guessing and logistic regression suggest that semiprimes and 3-almost primes possess statistical regularities that are considerably easier for machine learning algorithms to recognize than those associated with higher-order almost primes.

2 Number-Theoretic Motivation

Typical size of $\Omega(n)$. The distribution of $\Omega(n)$ is well understood. Hardy and Ramanujan [8] proved that the normal order of $\Omega(n)$ is $\log \log n$, while the Erdős–Kac theorem [4] showed that, after suitable normalization, $\Omega(n)$ is asymptotically Gaussian. Since $\log \log n$ grows extremely slowly, the values of $\Omega(n)$ remain concentrated in a relatively narrow range even for very large integers. For example, $\log \log(2^{44}) \approx 3.42$ and $\log \log(2^{176}) \approx 4.80$. Consequently, the overwhelming majority of integers in our four sampling windows satisfy $2 \leq \Omega(n) \leq 8$.

Indeed, the seven values $r = 2, \dots, 8$ account for 92.66% of the integers in the window near 2^{44} and 86.46% of those near 2^{176} (Appendix Tables 1–4). Although the proportion of integers with $\Omega(n) \geq 9$ increases from 7.34% to 13.54% across these scales, each individual class with $r \geq 9$ remains too sparse for reliable training under the natural distribution. We therefore restrict our study to the seven binary classification problems with $r = 2, \dots, 8$.

The classification problems. For each fixed value $r \in \{2, \dots, 8\}$, we consider the binary classification problem

$$\text{target class : } \Omega(n) = r, \quad \text{background class : } \Omega(n) \in \{2, \dots, 8\} \setminus \{r\}.$$

We refer to this as the *r-almost prime classification problem*. Each value of r defines an independent binary classification problem. The datasets are sampled directly from natural intervals without subsampling or class balancing, so that the target proportion π_r is simply the empirical frequency of $\Omega(n) = r$ in the corresponding interval.

Label convention. Throughout the paper the target class is assigned the label -1 and the background class the label $+1$. Since the background contains integers with several different values of $\Omega(n)$, including both even and odd parities, this labeling differs from the Liouville-sign convention used in Wang and Ye [19].

3 Integer-Vector Encoding

Modern machine learning libraries typically represent numerical features using `float32` or `float64` arithmetic. Consequently, integers larger than 2^{53} cannot be represented exactly in standard double precision, making it impossible to use large integers directly as input features. To overcome this limitation, Wang and Ye [19, 20] introduced *integer-vector encodings*, which map a large integer to a short vector of smaller integers while preserving sufficient arithmetic information for machine learning. Throughout this paper we use two of their encodings.

Let $f_d(n) = (m_1, \dots, m_d)$ denote a d -dimensional encoding of the integer n . Each component m_i lies below 2^{53} and therefore can be represented exactly in `float64`. The machine learning algorithms operate on the feature vector $f_d(n)$ rather than on the original integer n .

Following [19], we fix $M = 2^{44}$, so that a d -dimensional encoding can represent integers up to $M^d = 2^{44d}$. We consider the following two mappings.

- **Digit encoding (e_d).** The components m_i are simply the base- M digits of n . Thus e_d represents the integer by its positional expansion in base 2^{44} .
- **Hash encoding (h_d).** Choose a fixed integer A_2 and four fixed primes p_1, p_2, p_3, p_4 lying in the interval $[2^{45}, 2^{47}]$. The components of h_d are obtained by computing $A_2 n$ and taking its remainders modulo the first d of these primes. Since every remainder is less than 2^{47} , each component is represented exactly in double precision.

The constant A_2 and the primes $p_1, \dots, p_4$ are identical to those used by [19, §4] and are listed in the appendix.

Training is performed on integers near 2^{44} using $d \in \{2, 3, 4\}$. The trained classifiers are then applied, without retraining, to integers near 2^{88} , 2^{132} , and 2^{176} , corresponding to encoding dimensions $d = 2, 3, 4$, respectively. At the training scale every integer occupies a single base- M block, and the two encodings exhibit nearly identical performance (Appendix Table 8). At larger scales, however, the integers span multiple base- M blocks, and the two encodings begin to exhibit noticeably different cross-scale behavior.

4 Construction of Datasets

The objective of this section is to describe the datasets and feature representations used throughout the computational experiments. To evaluate the ability of machine learning models to recognize r -almost primes across different integer scales, we construct four datasets centered near 2^{44} , 2^{88} , 2^{132} , and 2^{176} . The smallest dataset is used for training, while the remaining three serve exclusively as independent test sets for evaluating cross-scale generalization. Following [19], each dataset consists of consecutive integers immediately below the corresponding power of two.

For $k \in \{44, 88, 132, 176\}$, let $C_k = 2^k$, $c = 2^{22} - 1$, and define the sampling interval

$$I_{C_k, c} = \left\{ n \in \mathbb{Z} : 2^k - 2^{22} + 1 \leq n < 2^k \right\}.$$

Each interval therefore contains exactly 2^{22} consecutive integers immediately below 2^k . For every integer $n \in I_{C_k, c}$ we compute the exact value of $\Omega(n)$ using `primeomega` from the Python package `sympy` [15].

Because the four sampling windows are centered at widely separated scales, the intervals $I_{C_{44}, c}$, $I_{C_{88}, c}$, $I_{C_{132}, c}$, and $I_{C_{176}, c}$ are pairwise disjoint. Consequently, no integer used to train a model can appear in any of the cross-scale test sets.

As explained in Section 2, only integers satisfying $2 \leq \Omega(n) \leq 8$ are retained. The remaining integers, for which $\Omega(n) \geq 9$, form a relatively small tail of the natural distribution and are excluded from the present study. For each fixed $r \in \{2, \dots, 8\}$, we then construct the corresponding r -almost prime classification problem by taking the target class $\Omega(n) = r$ against the background class $\Omega(n) \in \{2, \dots, 8\} \setminus \{r\}$. The resulting class sizes are the natural frequencies occurring in the sampling interval (Appendix Tables 1–4); no subsampling or class balancing is performed.

Each integer is represented by a feature vector consisting of two components: remainder features and an integer-vector encoding. Following [19], for $j \in \{4, 6, 8, 16\}$, let Rem_j denote the feature set consisting of the integer together with its residues modulo the squares of the first j prime numbers. To avoid storing the original large integer directly, we replace it by one of the two integer-vector encodings introduced in Section 3, obtaining the feature set Rem_j^d . Each observation therefore consists of $j + d$ numerical features.

Consequently, every experiment in this paper is determined by five parameters:

1. the target value r ;
2. the integer-vector encoding (e_d or h_d);
3. the encoding dimension d ;
4. the number of remainder features j ; and
5. the testing scale 2^{44} , 2^{88} , 2^{132} , or 2^{176} .

For the cross-scale experiments, the encoding dimension is determined by the testing scale, $d = k/44$, so that $d = 2, 3$, and 4 are used at the testing scales 2^{88} , 2^{132} , and 2^{176} , respectively. Unless otherwise stated, Section 6 reports the principal experiments using the hash encoding h_4 with $j = 8$ remainder features, while the complete parameter sweep over both encodings and all values of j is given in the Appendix.

Same-scale performance is evaluated by stratified five-fold cross-validation on the dataset near 2^{44} . For cross-scale evaluation, the models trained at 2^{44} are applied directly, without retraining, to the three larger datasets. Because these larger datasets are used exclusively for testing, Appendix Tables 2–4 report only the total numbers of observations in each target class rather than train/test splits.

5 Models and Metrics

Random forests, introduced by Breiman [3], construct an ensemble of decision trees whose predictions are aggregated to improve classification accuracy and robustness. Random forests and logistic regression are standard supervised learning methods widely used in statistical learning [9].

In this work we compare two supervised learning methods: random forests (RF) and logistic regression (LR). Random forest classifiers are implemented using `RandomForestClassifier` from `pyspark` on the University of Iowa Argon high-performance computing cluster [18], with the same hyperparameter settings used by [19]: Gini impurity [5], $\sqrt{\# \text{features}}$ candidate features at each split, `numTrees` = 150, and `maxDepth` = 15. As a linear benchmark we train logistic regression classifiers using `LogisticRegression` from `pyspark`. Both RF and LR are trained under the same stratified five-fold cross-validation protocol.

Each random forest model consists of five independently trained forests, one for each fold. For same-scale evaluation, each fold is held out once and predicted by the forest trained on the remaining four folds, after which the five confusion matrices (cf. Ting [17]) are aggregated. For cross-scale evaluation, the five forests trained at the 2^{44} scale are applied directly, without retraining, to the entire test dataset at the larger scale. Their predictions are then combined by majority vote to produce a single confusion matrix. The logistic regression models follow the same evaluation protocol. For both RF and LR, the classification threshold is selected during training to maximize the performance metric P_4 and is kept fixed for all larger testing scales.

Let T denote the target class ($\Omega(n) = r$), B the background class, and C_T and C_B the corresponding predicted classes. From the resulting confusion matrix we compute the four standard classification measures

$$\begin{aligned} \text{Precision} &= P(T | C_T), \\ \text{Recall} &= P(C_T | T), \\ \text{Specificity} &= P(C_B | B), \\ \text{NPV} &= P(B | C_B), \end{aligned}$$

where NPV denotes the negative predictive value.

Following Sitarz [14], we summarize classifier performance by the harmonic mean

$$P_4 = \frac{4}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}} + \frac{1}{\text{Specificity}} + \frac{1}{\text{NPV}}},$$

which assigns equal weight to all four quantities and is invariant under interchanging the target and background classes. Throughout the paper we denote by P_4^{RF} and P_4^{LR} the values of the above metric obtained from the random forest and logistic regression classifiers, respectively.

Because our datasets follow the natural distribution of r -almost primes rather than an artificially balanced construction, the appropriate reference is the *natural random-guessing baseline*.

Let

$$\pi_r = \frac{|\{n \in I_{C_k, c} : \Omega(n) = r\}|}{|\{n \in I_{C_k, c} : 2 \leq \Omega(n) \leq 8\}|}$$

denote the empirical proportion of the target class at testing scale 2^k . Consider a classifier that predicts the target class independently of the input with probability π_r . Such a classifier satisfies

$$\begin{aligned} \text{Precision} &= \text{Recall} = \pi_r, \\ \text{Specificity} &= \text{NPV} = 1 - \pi_r. \end{aligned}$$

Its corresponding P_4 value is therefore

$$P_4^{RG} = \frac{4}{2\pi_r^{-1} + 2(1 - \pi_r)^{-1}} = 2\pi_r(1 - \pi_r),$$

which we call the *natural random-guessing baseline*.

When $\pi_r = 1/2$, the classes are balanced and $P_4^{RG} = 1/2$. Under the natural distribution considered here, however, π_r depends on both r and the testing scale. Consequently, the natural random-guessing baseline P_4^{RG} also varies from one experiment to another. For this reason, comparisons across different values of r should be interpreted relative to P_4^{RG} rather than in terms of the absolute value of P_4 .

6 Results

6.1 Same-scale results

Table 1 summarizes the same-scale performance of the random forest (RF) and logistic regression (LR) classifiers on the seven r -almost prime classification problems at the training scale 2^{44} , using the hash encoding h_4 with $j = 8$ remainder features. The first row reports the corresponding values of the natural random-guessing baseline P_4^{RG} . Since the empirical target proportion π_r varies considerably with r , the baseline ranges from 0.079 at $r = 8$ to 0.355 at $r = 4$. Consequently, comparisons across different values of r should be interpreted relative to P_4^{RG} rather than to the balanced-class value $1/2$.

Appendix Table 5 reports the corresponding confusion matrices. Throughout this section we fix $j = 8$. Appendix Table 7 shows that P_4^{LR} is essentially insensitive to the choice of j , varying by less than 10^{-3} for every r , whereas P_4^{RF} changes by less than 0.03 except at $r = 7$, where it decreases from 0.562 at $j = 4$ to 0.433 at $j = 16$. Accordingly, the discussion below concerns the principal experiments with $j = 8$.

Three principal observations emerge.

1. Both classifiers consistently outperform the natural random-guessing baseline. The improvement $\Delta^{RF} = P_4^{RF} - P_4^{RG}$ ranges from 0.170 to 0.489, while $\Delta^{LR} = P_4^{LR} - P_4^{RG}$ ranges from 0.108 to 0.376. Thus both RF and LR successfully learn arithmetic information from the feature vectors for every $r = 2, \dots, 8$, although the amount of information captured depends strongly on the target value r .
2. Random forests outperform logistic regression uniformly at the training scale. The difference $P_4^{RF} - P_4^{LR}$ is positive for every r , ranging from 0.026 at $r = 7$ to 0.156 at $r = 2$. Thus the nonlinear RF model consistently provides the strongest same-scale classifier.
3. Among the seven binary classification problems, $r = 7$ is the most challenging. At $j = 8$, both classifiers attain their smallest P_4 values at $r = 7$. The corresponding confusion matrices, however, reveal markedly different decision strategies. The RF classifier emphasizes specificity at the expense of recall, whereas LR produces a more balanced trade-off. This contrast is illustrated below.

We conclude this subsection with two representative examples: $r = 2$, where RF achieves its best same-scale performance, and $r = 7$, where both classifiers attain their smallest P_4 values.

Representative example: $r = 2$. At $r = 2$, Appendix Table 5 shows that the RF classifier achieves high recall (0.686), specificity (0.807), and negative predictive value (0.946), while maintaining moderate precision (0.343). Collectively, these measures yield the largest same-scale performance, $P_4^{RF} = 0.600$, among the seven r -almost prime classification problems. In contrast, LR recalls fewer than half of the target integers while achieving only slightly lower specificity. This difference in target recall accounts for the largest gap between the two classifiers, $P_4^{RF} - P_4^{LR} = 0.156$.

Representative example: $r = 7$. At $r = 7$, both classifiers produce similar overall performance, $P_4^{RF} = 0.446$ and $P_4^{LR} = 0.420$, yet the confusion matrices reveal fundamentally different prediction strategies. The RF classifier is highly conservative, predicting the target class only rarely. Consequently, its specificity reaches 0.966, whereas its recall falls to 0.242. The LR classifier adopts a much less conservative strategy, achieving substantially higher recall (0.470) at the expense of lower specificity (0.843). Although the two classifiers arrive at nearly the same P_4 , they do so through markedly different balances between sensitivity and specificity.

6.2 Cross-scale results

The remaining three panels of Table 1 report the performance of the classifiers trained at the 2^{44} scale when they are applied, without retraining, to integers near 2^{88} , 2^{132} , and 2^{176} . Appendix Table 6 gives the corresponding confusion matrices. Because the empirical target proportion π_r changes with the testing scale (Section 2), the natural random-guessing baseline P_4^{RG} also changes from panel to panel. Accordingly, cross-scale performance should again be interpreted relative to P_4^{RG} rather than by comparing absolute values of P_4 .

Three principal observations emerge.

1. The random forest classifier exhibits genuine cross-scale generalization only for $r = 2$ and $r = 3$. For $r = 3$, it achieves the largest P_4 value in every cross-scale panel, namely 0.486, 0.455, and 0.425, while remaining approximately 0.22 above the natural random-guessing baseline throughout. For $r = 2$, P_4^{RF} remains remarkably stable (0.342, 0.322, 0.333), whereas P_4^{RG} decreases from 0.145 to 0.091 as the target class becomes increasingly rare. Consequently, the improvement $P_4^{RF} - P_4^{RG}$ actually increases with the testing scale, making $r = 2$ the strongest example of durable cross-scale generalization.
2. For the intermediate values $r = 4, 5$, and 7 , logistic regression provides the stronger cross-scale classifier. At every testing scale, $P_4^{LR} > P_4^{RF}$, and P_4^{LR} remains above the natural random-guessing baseline for $r = 2, \dots, 7$, falling below it only for $r = 8$ at 2^{176} . These results suggest that the nonlinear structure learned by the random forest at the training scale does not transfer as effectively to these intermediate classification problems.
3. The choice of integer-vector encoding has little influence at the training scale but becomes increasingly important for cross-scale prediction. Appendix Table 8 shows that the difference between the digit encoding and the hash encoding widens substantially on larger integers. The largest discrepancy occurs for logistic regression at $r = 8$, where P_4^{LR} increases from 0.151 under h_4 to 0.405 under e_4 at the testing scale 2^{88} . Thus the robustness of a classifier across scales depends not only on the learning algorithm but also on the choice of integer-vector representation.

We conclude with one representative example illustrating the different cross-scale behavior of the two classifiers.

Representative example: $r = 5$. At the training scale, the random forest classifier outperforms logistic regression, $P_4^{RF} = 0.511$ and $P_4^{LR} = 0.448$. After transferring the models to larger integers, however, their roles reverse. The random forest classifier drops to 0.225, 0.231, 0.225, at the three testing scales, remaining approximately 0.10 below the corresponding natural random-guessing baselines. Appendix Table 6 shows that this deterioration is caused primarily by a collapse in target recall, which decreases from 0.434 at the training scale to approximately 0.10 on all three larger datasets, while specificity increases to about 0.85–0.89. In contrast, logistic regression maintains $P_4^{LR} = 0.358, 0.378, 0.363$, remaining slightly above P_4^{RG} at every testing scale. Thus the same-scale advantage of random forests disappears completely, and logistic regression becomes the more robust classifier for $r = 5$ under cross-scale transfer.

Test scale	Model	$r=2$	$r=3$	$r=4$	$r=5$	$r=6$	$r=7$	$r=8$
2^{44}	RG	0.224	0.337	0.355	0.303	0.219	0.138	0.079
	LR	0.444	0.515	0.463	0.448	0.441	0.420	0.455
	RF	0.600	0.575	0.525	0.511	0.526	0.446	0.568
2^{88}	RG	0.145	0.270	0.333	0.327	0.274	0.198	0.127
	LR	0.268	0.402	0.436	0.358	0.388	0.395	0.151
	RF	0.342	0.486	0.275	0.225	0.446	0.288	0.187
2^{132}	RG	0.111	0.230	0.313	0.331	0.298	0.233	0.161
	LR	0.255	0.372	0.430	0.378	0.408	0.405	0.181
	RF	0.322	0.455	0.258	0.231	0.400	0.275	0.175
2^{176}	RG	0.091	0.205	0.296	0.330	0.311	0.256	0.186
	LR	0.187	0.351	0.421	0.363	0.387	0.373	0.150
	RF	0.333	0.425	0.338	0.225	0.378	0.261	0.186

Table 1: P_4 metric for LR and RF classifiers trained near 2^{44} and tested at 2^{44} , 2^{88} , 2^{132} , and 2^{176} , under the h_4 mapping with $j = 8$ remainder features. The RG row reports $2\pi_r(1 - \pi_r)$ at each test scale, where π_r is the empirical target density.

7 Computational Performance

All experiments were performed on the University of Iowa Argon high-performance computing cluster [18]. Training was carried out using stratified five-fold cross-validation on the dataset near 2^{44} , while cross-scale prediction applied the trained models, without retraining, to the three larger datasets near 2^{88} , 2^{132} , and 2^{176} . Table 2 summarizes the wall-clock training time, prediction time, and peak memory usage for the random forest (RF) and logistic regression (LR) models under both integer-vector encodings.

Random forest training required substantially more computational resources than logistic regression. Across the 21 (r, d) combinations for each model–encoding pair, median RF training time was approximately 6.3–6.4 hours, compared with approximately 0.20–0.23 hours for LR. Median peak memory usage was about 53 GB for RF and 27–31 GB for LR. These measurements reflect the computational cost of fitting the five-fold classifiers over the full set of remainder-feature choices $j \in \{4, 6, 8, 16\}$.

Cross-scale prediction was evaluated on the complete natural-distribution test windows near 2^{88} , 2^{132} , and 2^{176} , each containing approximately 3.6–3.8 million integers satisfying $2 \leq \Omega(n) \leq 8$.

Median prediction time per test set was approximately 2 minutes for RF and 29 seconds for LR. Equivalently, the median per-integer prediction time was approximately 30–40 μs for RF and about 8 μs for LR. Thus, once training is complete, both methods can classify millions of large integers rapidly, with logistic regression requiring substantially less inference time.

For comparison, Wang and Ye [19, Section 7] reported approximately 86.4 hours to compute the exact values of $\Omega(n)$ for the 2^{22} integers in the window near 2^{176} , corresponding to approximately 74 200 μs per integer. Although the two approaches solve different computational problems, this comparison illustrates the substantial speed advantage of machine-learning inference once a classifier has been trained. The RF and LR classifiers provide probabilistic predictions rather than exact values of $\Omega(n)$, but their per-integer inference cost is several orders of magnitude smaller than that of exact factor-count computation in this comparison.

Model	Encoding	Training time (h)	Peak memory (GB)	Prediction time ($\mu\text{s}/\text{integer}$)
RF	e_d	6.31 (1.52–9.59)	53.2 (52.4–91.5)	30.4 (11.8–170.0)
RF	h_d	6.41 (1.78–11.35)	53.2 (52.5–64.6)	40.0 (15.9–193.6)
LR	e_d	0.20 (0.17–0.27)	26.9 (23.2–48.9)	7.6 (6.3–21.5)
LR	h_d	0.23 (0.16–0.30)	31.4 (24.2–46.5)	7.7 (6.1–16.2)

Table 2: Computational performance of RF and LR classifiers. Entries report the median, with the observed range in parentheses, across the 21 (r, d) combinations for each model–encoding pair.

Overall, the computational results indicate that the integer-vector encoding framework is practical for large-scale probabilistic recognition of almost primes. The principal computational cost lies in model training, particularly for random forests, whereas prediction requires only a few tens of microseconds per integer. The present experiments were constrained by the available hardware and the use of standard implementations; larger-memory systems, additional parallelization, or more specialized implementations may therefore permit training on substantially larger datasets and extension to even larger integer scales.

8 Conclusion and Discussion

We have extended the integer-vector encoding framework of Wang and Ye [19, 20] from semiprime recognition to the seven r -almost prime classification problems $r = 2, \dots, 8$, under the natural distribution of almost primes. Using integer-vector encodings together with remainder features, we trained random forest (RF) and logistic regression (LR) classifiers on integers near 2^{44} and evaluated their performance both at the training scale and on substantially larger integers near 2^{88} , 2^{132} , and 2^{176} , without retraining.

The experiments lead to three principal conclusions.

First, random forest classifiers can be trained successfully for all seven binary classification problems. At the training scale, P_4^{RF} consistently exceeds both the logistic regression benchmark P_4^{LR} and the natural random-guessing baseline P_4^{RG} , demonstrating that the feature representations contain useful arithmetic information for every $r = 2, \dots, 8$. Thus machine learning is capable of distinguishing r -almost primes from other integers with $2 \leq \Omega(n) \leq 8$ substantially better than random guessing, even under the natural class distribution.

Second, genuine cross-scale generalization is observed only for the classifiers trained to recognize semiprimes and 3-almost primes. Without retraining, these classifiers continue to outperform both P_4^{RG} and P_4^{LR} when applied to integers that are many orders of magnitude larger than those used for training. For higher-order almost primes, however, the predictive advantage of random forests

diminishes rapidly, indicating that the arithmetic patterns learned at the 2^{44} scale do not transfer robustly across scales.

Third, the experiments suggest that semiprimes and 3-almost primes possess statistical regularities that are more readily captured by machine learning than those associated with higher-order almost primes. This conclusion is supported by the consistently larger values of $P_4^{RF} - P_4^{RG}$ for $r = 2$ and $r = 3$ across all testing scales. From the viewpoint of probabilistic computation, these results indicate that semiprimes and 3-almost primes are intrinsically easier to recognize probabilistically than r -almost primes with $r \geq 4$.

These findings should be interpreted in the context of existing deterministic algorithms. Primality testing is solvable in deterministic polynomial time by the AKS algorithm [1]. The present work addresses a fundamentally different problem: the probabilistic recognition of r -almost primes without explicit factorization. Our classifiers do not provide certificates of correctness, nor do they replace deterministic algorithms. Rather, they demonstrate that machine learning can exploit statistical structure in the arithmetic of integers to identify certain classes of almost primes with nontrivial accuracy.

The experiments also raise an intriguing number-theoretic question. Why do classifiers trained to recognize semiprimes and 3-almost primes exhibit substantially stronger cross-scale generalization than those trained for higher-order almost primes? Is this phenomenon merely a consequence of the present feature representation, or does it reflect underlying arithmetic structure? In particular, it would be of considerable interest to determine whether integers satisfying $\Omega(n) = 2$ or $\Omega(n) = 3$ possess structural properties that distinguish them, from the perspective of probabilistic recognition, from integers with $\Omega(n) \geq 4$.

Disclosure statement

The authors report there are no competing interests to declare.

AI Use Statement

Generative artificial intelligence (AI) was used solely to assist with language editing, improving clarity, and refining the organization and presentation of the manuscript. Specifically, ChatGPT (OpenAI, GPT-5.5) was used to suggest revisions to the writing and presentation of the text. The AI tool was not used to generate the research questions, mathematical results, computational experiments, data analyses, interpretations, or conclusions. All scientific content, mathematical arguments, experimental design, and final editorial decisions were developed, verified, and approved by the authors.

Code availability

Code used in this study is available from the corresponding author upon reasonable request.

References

- [1] Agrawal, M., Kayal, N., and Saxena, N. PRIMES is in P. *Annals of Mathematics*, **160**(2): 781–793, 2004. doi:[10.4007/annals.2004.160.781](https://doi.org/10.4007/annals.2004.160.781).

- [2] Booker, A., Hiary, G., and Keating, J. Detecting squarefree numbers. *Duke Mathematical Journal*, **164**(2):235–275, 2015. doi:[10.1215/00127094-2856619](https://doi.org/10.1215/00127094-2856619).
- [3] Breiman, L. Random forests. *Machine Learning*, **45**(1):5–32, 2001. doi:[10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [4] Erdős, P. and Kac, M. The Gaussian law of errors in the theory of additive number theoretic functions. *American Journal of Mathematics*, **62**(1/4):738–742, 1940. doi:[10.2307/2371483](https://doi.org/10.2307/2371483).
- [5] Fürnkranz, J. Decision tree. In C. Sammut and G. I. Webb, editors, *Encyclopedia of Machine Learning*. Springer, 2010. doi:[10.1007/978-0-387-30164-8](https://doi.org/10.1007/978-0-387-30164-8).
- [6] Halberstam, H. and Richert, H.-E., *Sieve Methods*. Academic Press, 1974.
- [7] Hardy, G. H. and Wright, E. M. *An Introduction to the Theory of Numbers*. Oxford University Press, 6 edition, 2008. ISBN 978-0-19-921986-5.
- [8] Hardy, G. H. and Ramanujan, S. The normal number of prime factors of a number n . *Quarterly Journal of Mathematics*, **48**:76–92, 1917. URL <https://ramanujan.sirinudi.org/Volumes/published/ram35.html>.
- [9] Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, 2 edition, 2009. doi:[10.1007/978-0-387-84858-7](https://doi.org/10.1007/978-0-387-84858-7).
- [10] Landau, E. Sur quelques problèmes relatifs à la distribution des nombres premiers. *Bulletin de la Société Mathématique de France*, **28**:25–38, 1900. URL <http://eudml.org/doc/85978>.
- [11] Landau, E. *Handbuch der Lehre von der Verteilung der Primzahlen*. B. G. Teubner, Leipzig, 1909.
- [12] Qin, H. and Ye, Y. Algorithms of the Möbius function by random forests and neural networks. *Journal of Big Data*, **11**(31), 2024. doi:[10.1186/s40537-024-00889-7](https://doi.org/10.1186/s40537-024-00889-7).
- [13] Rivest, R., Shamir, A., and Adleman, L. A method for obtaining digital signatures and public-key cryptography. *Communications of the ACM*, **21**(2):120–126, 1978. doi:[10.1145/359340.359342](https://doi.org/10.1145/359340.359342).
- [14] Sitarz, M. Extending F1 metric, probabilistic approach. *Advances in Artificial Intelligence and Machine Learning*, **3**(2):1025–1038, 2023. doi:[10.54364/AAIML.2023.1161](https://doi.org/10.54364/AAIML.2023.1161).
- [15] SymPy Development Team. Number theory API, sympy documentation, version 1.13.3. <https://docs.sympy.org/latest/modules/nttheory.html>, 2024.
- [16] Tenenbaum, G. *Introduction to Analytic and Probabilistic Number Theory*, volume 163 of *Graduate Studies in Mathematics*. American Mathematical Society, 3 edition, 2015. doi:[10.1090/gsm/163](https://doi.org/10.1090/gsm/163).
- [17] Ting, K. Confusion matrix. In C. Sammut and G. I. Webb, editors, *Encyclopedia of Machine Learning*. Springer, 2010. doi:[10.1007/978-0-387-30164-8](https://doi.org/10.1007/978-0-387-30164-8).
- [18] University of Iowa. Argon: High performance computing services. <https://uiowa.atlassian.net/wiki/spaces/hpcdocs/overview>, 2025.

- [19] Wang, K. and Ye, Y. Computing the Liouville function by random forests on high-precision data via integer-vector encoding. Preprint, 2025.
- [20] Ye, Y. and Wang, K. Systems and methods for integer-vector encoding of high-precision data for machine learning, 2025. U.S. Provisional Patent Application No. 63/879,503, filed September 10, 2025.